02. Lighthouse 전체 아키텍처 개요

개요

Lighthouse는 Rust로 작성된 Ethereum 2.0 (합의 레이어) 클라이언트로, 복잡한 분산 시스템 아키텍처를 가지고 있습니다. 이 문서는 Lighthouse의 전체 구조와 주요 컴포넌트들의 역할, 그리고 실제 코드베이스에서의 구현을 체계적으로 분석합니다.

전체 시스템 아키텍처

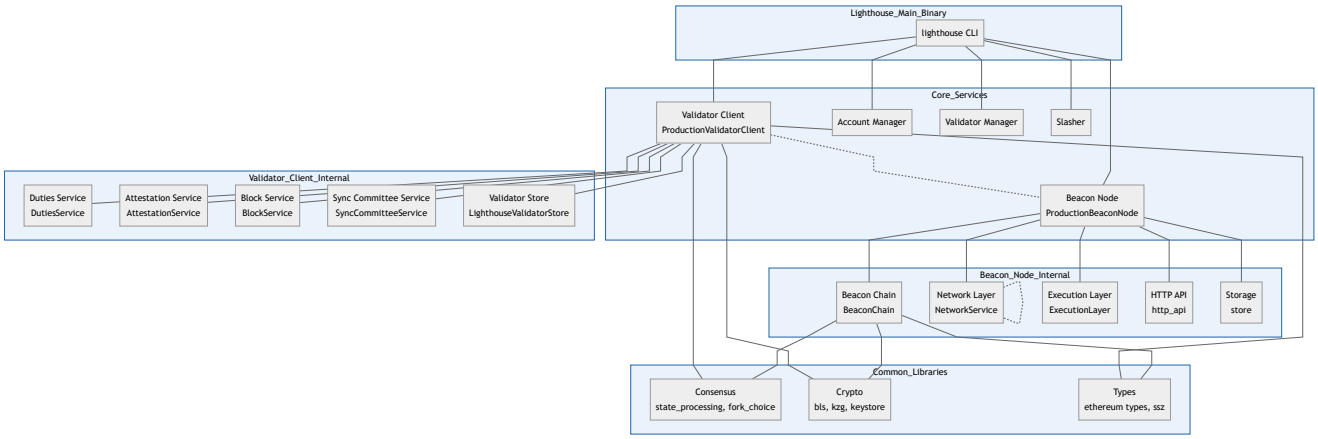
최상위 구조

Lighthouse는 Cargo 워크스페이스로 구성되어 있으며, 다음과 같은 주요 바이너리와 라이브러리들로 구성됩니다:

```
lighthouse/
├── lighthouse/ # 메인 바이너리 (CLI 진입점)
├── beacon_node/ # 비콘 노드 구현
├── validator_client/ # 검증자 클라이언트 구현
├── account_manager/ # 계정 관리 도구
├── validator_manager/ # 검증자 관리 도구
├── boot_node/ # 부트스트랩 노드
├── database_manager/ # 데이터베이스 관리 도구
├── slasher/ # 슬래싱 감지 서비스
├── common/ # 공통 라이브러리들
```

```
├── consensus/ # 합의 알고리즘 구현
├── crypto/ # 암호학 라이브러리들
└── testing/ # 테스트 도구들
```

핵심 컴포넌트 관계도



주요 컴포넌트 상세 분석

1. 메인 바이너리 (lighthouse/)

파일 위치: `lighthouse/src/main.rs`

메인 바이너리는 모든 Lighthouse 서비스의 진입점 역할을 합니다:

```rust
// lighthouse/src/main.rs에서 발췌
pub struct ProductionBeaconNode<E: EthSpec>(ProductionClient<E>);
```

주요 기능:

- CLI 인터페이스 제공
- 서브커맨드 라우팅 (beacon_node, validator_client, account_manager 등)
- 환경 설정 및 로깅 초기화
- 네트워크 설정 (mainnet, testnet 등)

지원하는 서브커맨드:

- `beacon_node` : 비콘 노드 실행
- `validator_client` : 검증자 클라이언트 실행
- `account_manager` : 검증자 계정 관리
- `validator_manager` : 검증자 관리
- `boot_node` : 부트스트랩 노드 실행

2. 비콘 노드 (beacon_node/)

핵심 구조체: `ProductionBeaconNode<E: EthSpec>`

비콘 노드는 Ethereum 2.0 합의 프로토콜의 핵심 구현체입니다.

2.1 주요 서브모듈

beacon_chain: 합의 상태 관리

```
// beacon_node/beacon_chain/src/beacon_chain.rs
pub struct BeaconChain<T: BeaconChainTypes> {
pub spec: ChainSpec,
pub store: Arc<T::BeaconChainStore>,
pub slot_clock: T::SlotClock,
pub fork_choice: RwLock<ForkChoice<...>>,
// ... 기타 필드들
}
```

lighthouse_network: P2P 네트워킹

```
// beacon_node/lighthouse_network/src/service/mod.rs
pub struct NetworkService<E: EthSpec> {
libp2p: LibP2PService<E>,
gossipsub_processor: GossipProcessor<E>,
// ... 기타 필드들
}
```

execution_layer: 실행 레이어 통신

- Engine API를 통한 실행 클라이언트와의 통신
- 블록 빌딩 및 페이로드 처리

http_api: REST API 서버

- Beacon API 표준 구현
- 검증자 클라이언트와의 HTTP 통신

store: 데이터 저장소

- LevelDB/RocksDB 기반 영구 저장소
- 블록, 상태, 체크포인트 저장

2.2 초기화 과정

```
// beacon_node/src/lib.rs에서 발췌
impl<E: EthSpec> ProductionBeaconNode<E> {
pub  async  fn  new(
context: RuntimeContext<E>,
mut  client_config: ClientConfig,
) -> Result<Self, String> {
let  builder = ClientBuilder::new(context.eth_spec_instance.clone())
.runtime_context(context)
.chain_spec(spec.clone())
.beacon_processor(client_config.beacon_processor.clone())
.http_api_config(client_config.http_api.clone())
.disk_store(&db_path, &freezer_db_path, &blobs_db_path, store_config)?;
// ... 빌더 패턴을 통한 초기화
}
```

```
}
```

3. 검증자 클라이언트 (validator_client/)

핵심 구조체: `ProductionValidatorClient<E: EthSpec>`

검증자 클라이언트는 검증자 역할을 수행하는 독립적인 서비스입니다.

3.1 주요 서비스들

```rust
// validator_client/src/lib.rs에서 발췌
pub struct ProductionValidatorClient<E: EthSpec> {
duties_service: Arc<DutiesService<ValidatorStore<E>, SystemTimeSlotClock>>,
block_service: BlockService<ValidatorStore<E>, SystemTimeSlotClock>,
attestation_service: AttestationService<ValidatorStore<E>, SystemTimeSlotClock>,
sync_committee_service: SyncCommitteeService<ValidatorStore<E>, SystemTimeSlotCl
validator_store: Arc<ValidatorStore<E>>,
// ... 기타 필드들
}
```

DutiesService: 검증자 듀티 관리

- 비콘 노드로부터 듀티 정보 조회
- 증명자(attester) 및 제안자(proposer) 듀티 스케줄링

AttestationService: 증명 생성 및 제출

- 증명 데이터 생성
- BLS 서명 및 네트워크 전파

BlockService: 블록 제안

- 블록 생성 및 서명
- 실행 페이로드 포함

SyncCommitteeService: 동기화 위원회 참여

- 동기화 메시지 생성 및 집계

3.2 검증자 저장소

```
type ValidatorStore<E> = LighthouseValidatorStore<SystemTimeSlotClock, E>;
```

주요 기능:

- 검증자 키 관리
- 슬래싱 보호
- 서명 작업 수행

4. 공통 라이브러리들

4.1 합의 라이브러리 (consensus/)

state_processing: 상태 전이 함수들

- 슬롯 처리, 에포크 처리
- 블록 및 증명 처리

fork_choice: 포크 선택 알고리즘

- LMD-GHOST 구현
- ProtoArray 데이터 구조

types: 합의 관련 데이터 타입들

- BeaconState, BeaconBlock, Attestation 등

- SSZ 직렬화 지원

4.2 암호학 라이브러리 (crypto/)

bls: BLS 서명 및 검증

```
// BLS 서명 인터페이스 예시
pub trait Signature {
fn verify(&self, msg: &[u8], pubkey: &PublicKey) -> bool;
fn aggregate(signatures: &[Self]) -> Self;
}
```

kzg: KZG 커밋먼트 및 증명

- 블롭 데이터 처리
- 다항식 커밋먼트

eth2_keystore: 키스토어 관리

- EIP-2335 키스토어 형식
- 키 파생 및 저장

4.3 공통 유틸리티 (common/)

slot_clock: 슬롯 시간 관리

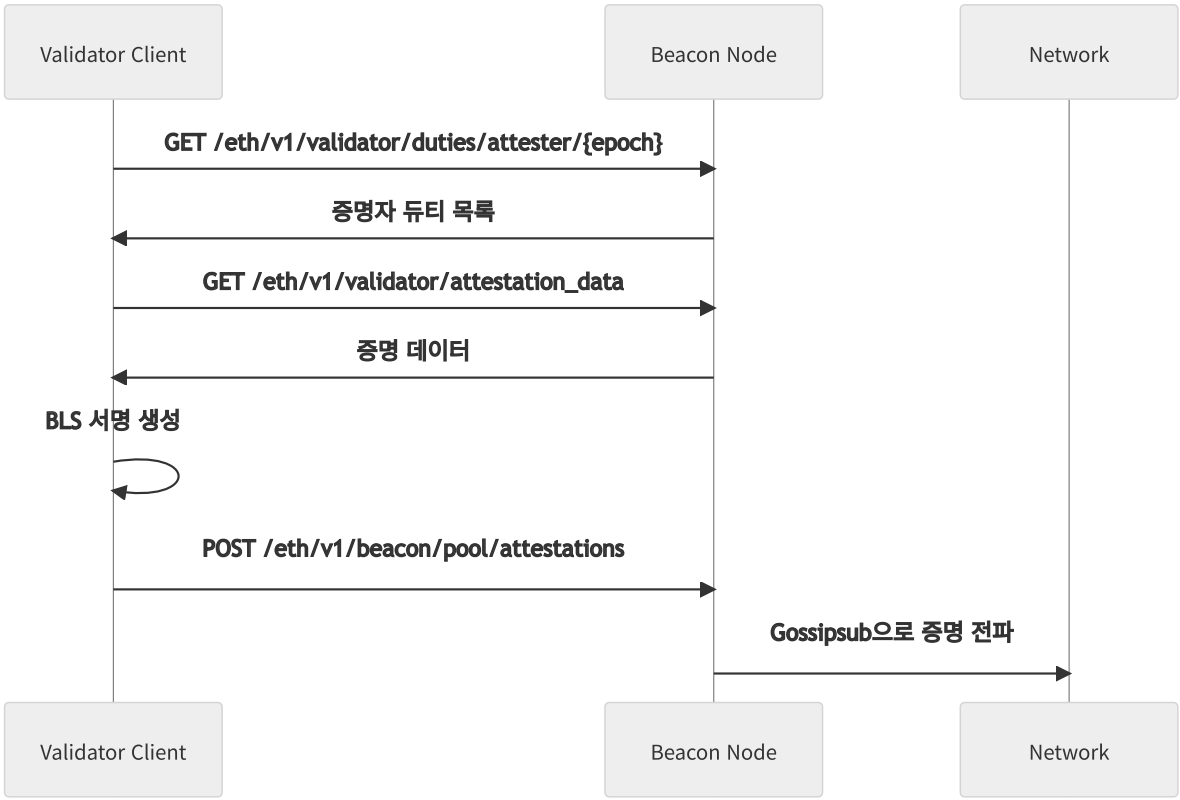
task_executor: 비동기 작업 실행

metrics: 프로메테우스 메트릭

logging: 구조화된 로깅

데이터 흐름 및 상호작용

1. 비콘 노드 ↔ 검증자 클라이언트



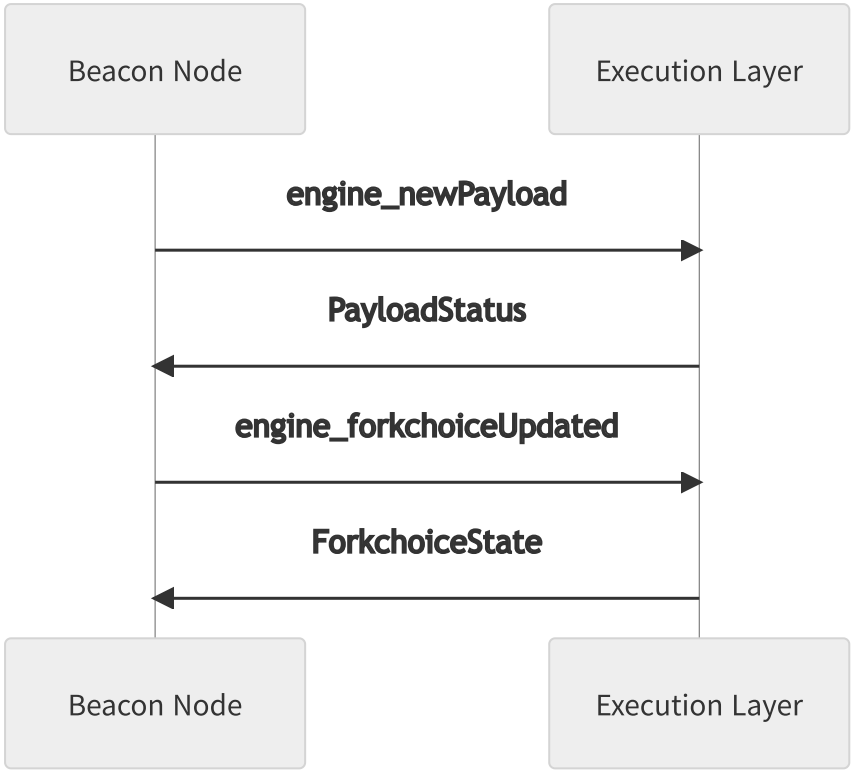
2. P2P 네트워크 통신

Gossipsub 토픽들:

- `beacon_block` : 블록 전파
- `beacon_attestation` : 증명 전파
- `voluntary_exit` : 자발적 탈퇴
- `proposer_slashing` : 제안자 슬래싱
- `attester_slashing` : 증명자 슬래싱

3. 실행 레이어 통신

비콘 노드는 Engine API를 통해 실행 클라이언트와 통신합니다:



설정 및 환경 관리

네트워크 설정

Lighthouse는 다음 네트워크들을 지원합니다:

- **mainnet**: 메인넷
- **holesky**: 테스트넷
- **sepolia**: 테스트넷
- **gnosis**: 그노시스 체인 (별도 빌드 필요)

데이터 디렉토리 구조

```
~/.lighthouse/{network}/
├── beacon/
```

```
| ├── chain_db/ # 체인 데이터베이스
| ├── freezer_db/ # 과거 상태 저장소
| └── network/ # 네트워크 키
└── validators/
├── validator_definitions.yml
├── slashing_protection.sqlite
└── validator_keys/
```

성능 및 최적화

메모리 관리

```
// lighthouse/src/main.rs에서 발췌
if  is_beacon_node && !matches.get_flag(DISABLE_MALLOC_TUNING_FLAG) {
if  let Err(e) = configure_memory_allocator() {
eprintln!("Unable to configure the memory allocator: {}", e);
exit(1)
}
}
```

- jemalloc 사용 (Windows 제외)
- 메모리 할당 최적화

병렬 처리

- Tokio 런타임 기반 비동기 처리
- Rayon을 통한 CPU 집약적 작업 병렬화
- 멀티스레드 블록 처리

모니터링 및 메트릭

HTTP 메트릭 서버

- 프로메테우스 형식 메트릭 제공
- 검증자 성능 추적
- 네트워크 상태 모니터링

로깅 시스템

- 구조화된 로깅 (tracing 크레이트)
- 로그 레벨별 필터링
- 파일 로테이션 지원

보안 고려사항

슬래싱 보호

- SQLite 기반 슬래싱 보호 데이터베이스
- 중복 서명 방지
- 검증자 키 격리

네트워크 보안

- libp2p 기반 보안 통신

- 피어 평판 시스템
- DDoS 방지 메커니즘

결론

Lighthouse는 모듈화된 아키텍처를 통해 Ethereum 2.0 합의 프로토콜을 구현합니다. 각 컴포넌트는 명확한 책임을 가지며, 표준화된 인터페이스를 통해 상호작용합니다. 이러한 설계는 유지보수성, 확장성, 그리고 보안성을 보장하며, 실제 프로덕션 환경에서의 안정적인 운영을 가능하게 합니다.

다음 문서에서는 각 컴포넌트의 구체적인 구현 세부사항을 더 깊이 있게 분석해보겠습니다.